

How to Build a Graph RAG App

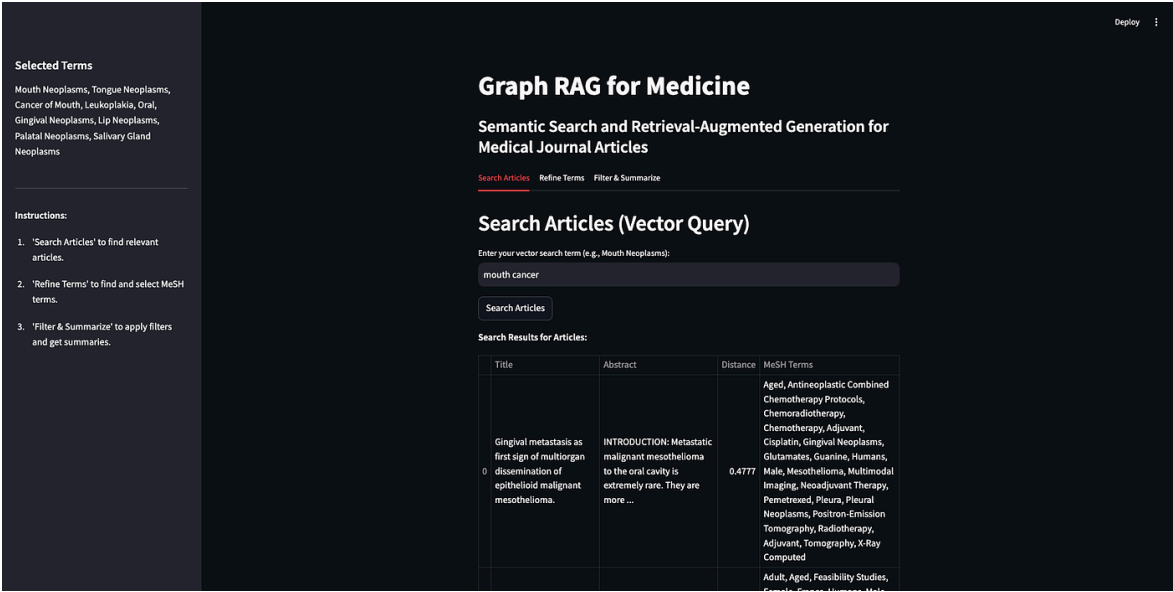


Image by Author

Using knowledge graphs and AI to retrieve, filter, and summarize medical journal articles

[



](https://medium.com/@stevehedden?source=post_page---byline---b323fc33ba06-----)

[



](https://medium.com/towards-data-science?source=post_page---byline--b323fc33ba06-----)

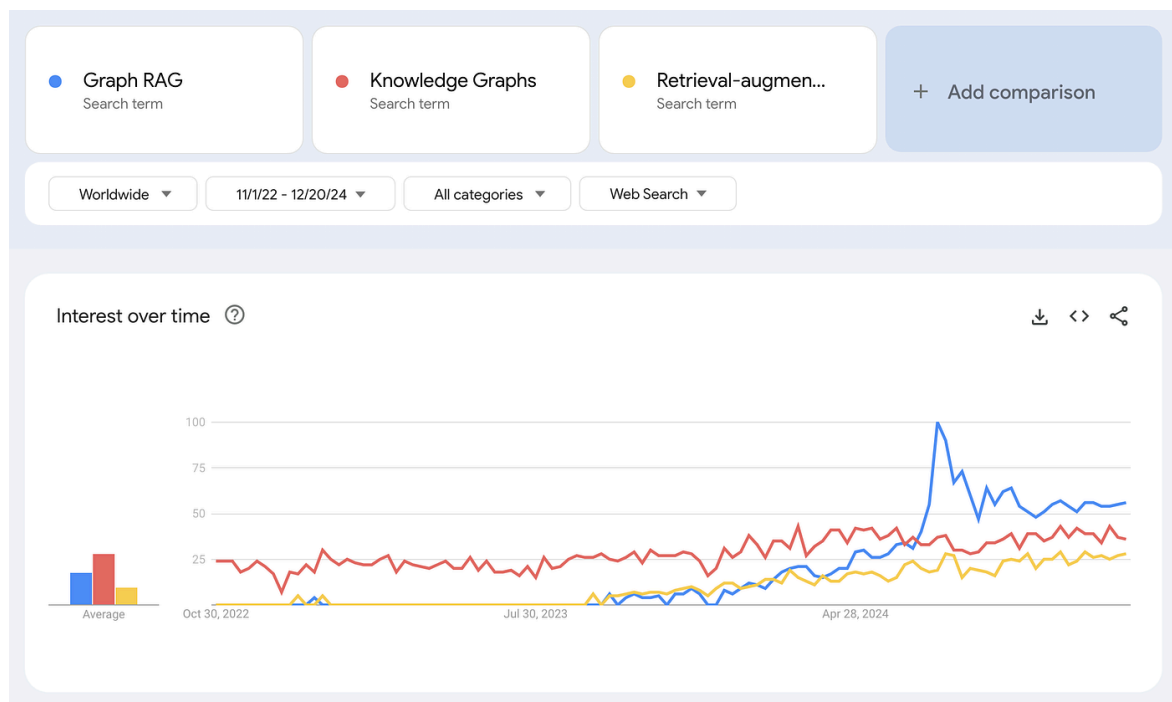
The accompanying code for the app and notebook are [here](#).

Knowledge graphs (KGs) and Large Language Models (LLMs) are a match made in heaven. My [previous](#) [posts](#) discuss the complementarities of these two technologies in more detail but the short version is, “some of the main weaknesses of LLMs, that they are black-box models and struggle with factual knowledge, are some of KGs’ greatest strengths. KGs are, essentially, collections of facts, and they are fully interpretable.”

This article is all about building a simple Graph RAG app. What is RAG? RAG, or Retrieval-Augmented Generation, is about **retrieving** relevant information to **augment** a prompt that is sent to an LLM, which **generates** a response. Graph RAG is RAG that uses a knowledge graph as part of the retrieval portion. If you’ve never heard of Graph RAG, or want a refresher, I’d watch [this video](#).

The basic idea is that, rather than sending your prompt directly to an LLM, which was not trained on your data, you can supplement your prompt with the relevant information needed for the LLM to answer your prompt accurately. The example I use often is copying a job description and my resume into ChatGPT to write a cover letter. The LLM is able to provide a much more relevant response to my prompt, ‘write me a cover letter,’ if I give it my resume and the description of the job I am applying for. Since knowledge graphs are built to store knowledge, they are a perfect way to store internal data and supplement LLM prompts with additional context, improving the accuracy and contextual understanding of the responses.

This technology has many, many, applications such [customer service bots](#), [drug discovery](#), [automated regulatory report generation in life sciences](#), [talent acquisition and management for HR](#), [legal research and writing](#), and [wealth advisor assistants](#). Because of the wide applicability and the potential to improve the performance of LLM tools, Graph RAG (that’s the term I’ll use here) has been blowing up in popularity. Here is a graph showing interest over time based on Google searches.



Source: <https://trends.google.com/>

Graph RAG has experienced a surge in search interest, even surpassing terms like knowledge graphs and retrieval-augmented generation. Note that Google Trends measures *relative* search interest, not absolute number of searches. The spike in July 2024 for searches of Graph RAG coincides with the week Microsoft [announced](#) that their GraphRAG application would be available on [GitHub](#).

The excitement around Graph RAG is broader than just Microsoft, however. Samsung acquired RDFox, a knowledge graph company, in July of 2024. The [article announcing that acquisition](#) did not mention Graph RAG explicitly, but in [this article in Forbes](#) published in November 2024, a Samsung spokesperson stated, "We plan to develop knowledge graph technology, one of the main technologies of personalized AI, and organically connect with generated AI to support user-specific services."

In October 2024, Ontotext, a leading graph database company, and Semantic Web company, the maker of PoolParty, a knowledge graph curation platform, merged to form [Graphwise](#). According to [the press release](#), the merger aims to "democratize the evolution of Graph RAG as a category."

While some of the buzz around Graph RAG may come from the broader excitement surrounding chatbots and generative AI, it reflects a genuine evolution in how knowledge graphs are being applied to solve complex, real-world problems. One example is that LinkedIn [applied Graph RAG](#) to improve

their customer service technical support. Because the tool was able to retrieve the relevant data (like previously solved similar tickets or questions) to feed the LLM, the responses were more accurate and the mean resolution time dropped from 40 hours to 15 hours.

This post will go through the construction of a pretty simple, but I think illustrative, example of how Graph RAG can work in practice. The end result is an app that a non-technical user can interact with. Like my last post, I will use a dataset consisting of medical journal articles from PubMed. The idea is that this is an app that someone in the medical field could use to do literature review. The same principles can be applied to many use cases however, which is why Graph RAG is so exciting.

The structure of the app, along with this post is as follows:

Step zero is preparing the data. I will explain the details below but the overall goal is to vectorize the raw data and, separately, turn it into an RDF graph. As long as we keep URIs tied to the articles before we vectorize, we can navigate across a graph of articles and a vector space of articles. Then, we can:

1. **Search Articles:** use the power of the vector database to do an initial search of relevant articles given a search term. I will use vector similarity to retrieve articles with the most similar vectors to that of the search term.
2. **Refine Terms:** explore the [Medical Subject Headings \(MeSH\) biomedical vocabulary](#) to select terms to use to filter the articles from step 1. This controlled vocabulary contains medical terms, alternative names, narrower concepts, and many other properties and relationships.
3. **Filter & Summarize:** use the MeSH terms to filter the articles to avoid 'context poisoning'. Then send the remaining articles to an LLM along with an additional prompt like, "summarize in bullets."

Some notes on this app and tutorial before we get started:

- This set-up uses knowledge graphs exclusively for metadata. This is only possible because each article in my dataset has already been tagged with terms that are part of a rich controlled vocabulary. I am using the graph for structure and semantics and the vector database for similarity-based retrieval, ensuring each technology is used for what it does best. Vector similarity can tell us "esophageal cancer" is semantically similar to "mouth cancer", but knowledge graphs can tell us the details of the relationship between "esophageal cancer" and "mouth cancer."

- The data I used for this app is a collection of medical journal articles from PubMed (more on the data below). I chose this dataset because it is structured (tabular) but also contains text in the form of abstracts for each article, and because it is already tagged with topical terms that are aligned with a well-established controlled vocabulary (MeSH). Because these are medical articles, I have called this app 'Graph RAG for Medicine.' But this same structure can be applied to any domain and is not specific to the medical field.
- What I hope this tutorial and app demonstrate is that you can improve the results of your RAG application in terms of accuracy and explainability by incorporating a knowledge graph into the retrieval step. I will show how KGs can improve the accuracy of RAG applications in two ways: by giving the user a way of filtering the context to ensure the LLM is only being fed the most relevant information; and by using domain specific controlled vocabularies with dense relationships that are maintained and curated by domain experts to do the filtering.
- What this tutorial and app don't directly showcase are two other significant ways KGs can enhance RAG applications: governance, access control, and regulatory compliance; and efficiency and scalability. For governance, KGs can do more than filter content for relevancy to improve accuracy — they can enforce data governance policies. For instance, if a user lacks permission to access certain content, that content can be excluded from their RAG pipeline. On the efficiency and scalability side, KGs can help ensure RAG applications don't die on the shelf. While it's easy to create an impressive one-off RAG app (that's literally the purpose of this tutorial), many companies struggle with a proliferation of disconnected POCs that lack a cohesive framework, structure, or platform. That means many of those apps are not going to survive long. A metadata layer powered by KGs can break down data silos, providing the foundation needed to build, scale, and maintain RAG applications effectively. Using a rich controlled vocabulary like MeSH for the metadata tags on these articles is a way of ensuring this Graph RAG app can be integrated with other systems and reducing the risk that it becomes a silo.

Step 0: Prepare the data

The code to prepare the data is in [this](#)  notebook.

As mentioned, I've again decided to use [this](#)  dataset of 50,000 research articles from the PubMed repository (License [CC0: Public Domain](#) ). This dataset contains the title of the articles, their abstracts, as well as a field for

metadata tags. These tags are from the Medical Subject Headings (MeSH) controlled vocabulary thesaurus. The PubMed articles are really just metadata on the articles — there are abstracts for each article but we don't have the full text. The data is already in tabular format and tagged with MeSH terms.

We can vectorize this tabular dataset directly. We could turn it into a graph (RDF) before we vectorize, but I didn't do that for this app and I don't know that it would help the final results for this kind of data. The most important thing about vectorizing the raw data is that we add [Unique Resource Identifiers](#) (URIs) to each article first. A URI is a unique ID for navigating RDF data and it is necessary for us to go back and forth between vectors and entities in our graph. Additionally, we will create a separate collection in our vector database for the MeSH terms. This will allow the user to search for relevant terms without having prior knowledge of this controlled vocabulary. Below is a diagram of what we are doing to prepare our data.

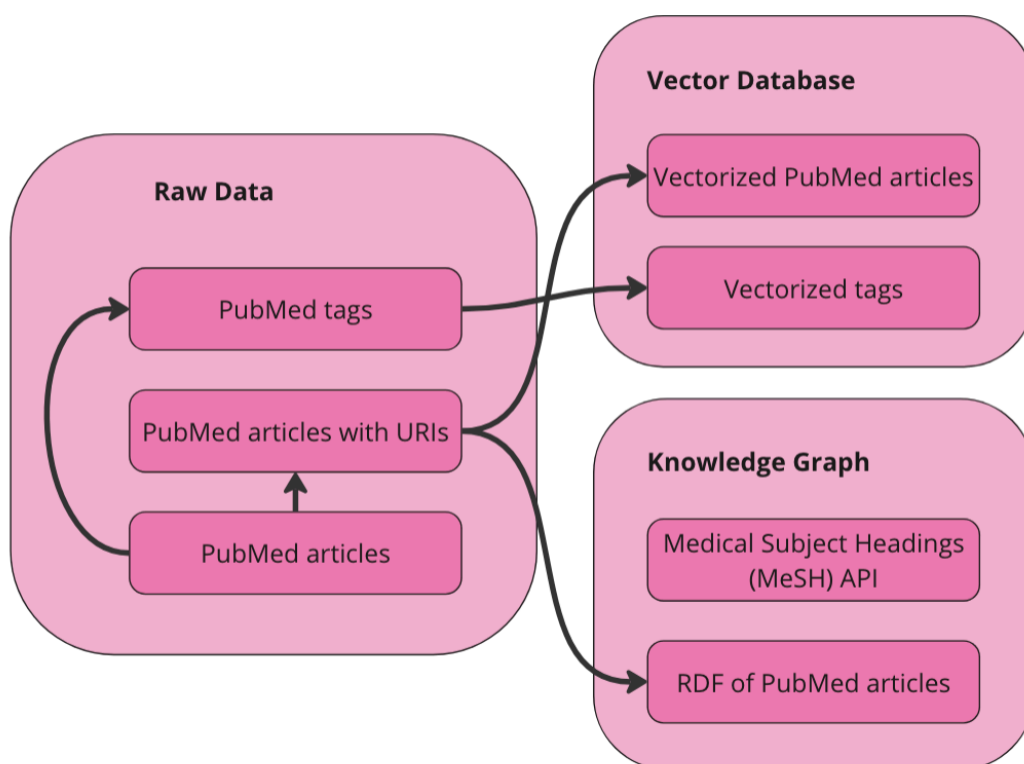


Image by Author

We have two collections in our vector database to query: articles and terms. We also have the data represented as a graph in RDF format. Since MeSH has an API, I am just going to query the API directly to get alternative names and narrower concepts for terms.

Vectorize data in Weaviate

First import the required packages and set up the Weaviate client:

```
import weaviate
from weaviate.util import generate_uuid5
from weaviate.classes.init import Auth
import os
import json
import pandas as pd
client = weaviate.connect_to_weaviate_cloud(
    cluster_url="XXX",
    auth_credentials=Auth.api_key("XXX"),
    headers={'X-OpenAI-API-key': "XXX"}
)
```

Read in the PubMed journal articles. I am using [Databricks](#) to run this notebook so you may need to change this, depending on where you run it. The goal here is just to get the data into a pandas DataFrame.

```
df = spark.sql("SELECT * FROM
workspace.default.pub_med_multi_label_text_classification_dataset_
processed").toPandas()
```

If you're running this locally, just do:

```
df = pd.read_csv("PubMed Multi Label Text Classification Dataset
Processed.csv")
```

Then clean the data up a bit:

```
import numpy as np

df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna('', inplace=True)
df['Title'] = df['Title'].astype(str)
df['abstractText'] = df['abstractText'].astype(str)
df['meshMajor'] = df['meshMajor'].astype(str)
```

Now we need to create a URI for each article and add that in as a new column. This is important because the URI is the way we can connect the vector representation of an article with the knowledge graph representation of the article.


```

import urllib.parse
from rdflib import Graph, RDF, RDFS, Namespace, URIRef, Literal
def create_valid_uri(base_uri, text):
    if pd.isna(text):
        return None

    sanitized_text = urllib.parse.quote(text.strip().replace(' ',
'_').replace('"', ' ').replace('<', ' ').replace('>',
' ').replace("'", "_"))
    return URIRef(f"{base_uri}/{sanitized_text}")
def create_article_uri(title,
base_namespace="http://example.org/article/"):
    """
    Creates a URI for an article by replacing non-word characters
    with underscores and URL-encoding.    Args:
        title (str): The title of the article.
        base_namespace (str): The base namespace for the article
    URI.    Returns:
        URIRef: The formatted article URI.
    """
    if pd.isna(title):
        return None

    sanitized_title = re.sub(r'\W+', '_', title.strip())

    sanitized_title = re.sub(r'_+', '_', sanitized_title)

    encoded_title = quote(sanitized_title)

    uri = f"{base_namespace}{encoded_title}"
    return URIRef(uri)
df['Article_URI'] = df['Title'].apply(lambda title:
create_valid_uri("http://example.org/article", title))

```

We also want to create a DataFrame of all of the MeSH terms that are used to tag the articles. This will be helpful later when we want to search for similar MeSH terms.

```

# Function to clean and parse MeSH terms
def parse_mesh_terms(mesh_list):
    if pd.isna(mesh_list):
        return []
    return [
        term.strip().replace(' ', '_')
        for term in mesh_list.strip("[]").split(',')
    ]
]# Function to create a valid URI for MeSH terms
def create_valid_uri(base_uri, text):
    if pd.isna(text):
        return None
    sanitized_text = urllib.parse.quote(
        text.strip()
        .replace(' ', '_')
        .replace('"', '')
        .replace('<', '')
        .replace('>', '')
        .replace("'", "_")
    )
    return f"{base_uri}/{sanitized_text}"# Extract and process all
MeSH terms
all_mesh_terms = []
for mesh_list in df["meshMajor"]:
    all_mesh_terms.extend(parse_mesh_terms(mesh_list))
Deduplicate terms
unique_mesh_terms = list(set(all_mesh_terms))# Create a DataFrame
of MeSH terms and their URIs
mesh_df = pd.DataFrame({
    "meshTerm": unique_mesh_terms,
    "URI": [create_valid_uri("http://example.org/mesh", term) for
term in unique_mesh_terms]
})# Display the DataFrame
print(mesh_df)

```

Vectorize the articles DataFrame:

```

from weaviate.classes.config import Configure
articles = client.collections.create(
    name = "Article",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(),
    generative_config=Configure.Generative.openai(),
)
articles = client.collections.get("Article")with
articles.batch.dynamic() as batch:
    for index, row in df.iterrows():
        batch.add_object({
            "title": row["Title"],
            "abstractText": row["abstractText"],
            "Article_URI": row["Article_URI"],
            "meshMajor": row["meshMajor"],
        })

```

Now vectorize the MeSH terms:

```

terms = client.collections.create(
    name = "term",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(),
    generative_config=Configure.Generative.openai(),
)
terms = client.collections.get("term")with terms.batch.dynamic()
as batch:
    for index, row in mesh_df.iterrows():
        batch.add_object({
            "meshTerm": row["meshTerm"],
            "URI": row["URI"],
        })

```

You can, at this point, run semantic search, similarity search, and RAG directly against the vectorized dataset. I won't go through all of that here but you can look at the code in my [accompanying notebook](#) to do that.

Turn data into a knowledge graph

I am just using the same code we used in the [last post](#) to do this. We are basically turning every row in the data into an "Article" entity in our KG. Then we are giving each of these articles properties for title, abstract, and MeSH terms. We are also turning every MeSH term into an entity as well. This code also adds random dates to each article for a property called date published and a random number between 1 and 10 to a property called access. We won't use those properties in this demo. Below is a visual representation of the graph we are creating from the data.

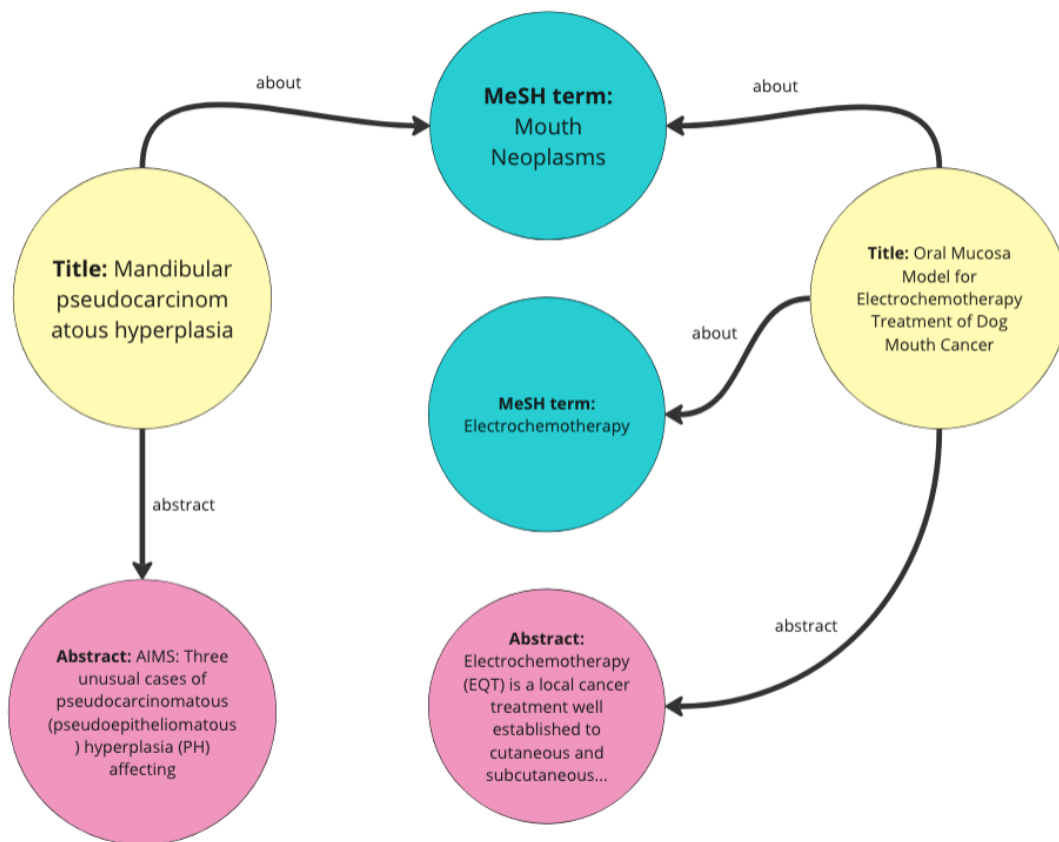


Image by Author

Here is how to iterate through the DataFrame and turn it into RDF data: